

Practical – 5

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Server

```
root@Rahul: ~/OSSP_FIFO_Sig × + v
GNU nano 7.2 server1.c *
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#define SERVER_FIFO "server_fifo"
#define CLIENT_FIFO "client_fifo"
int main() {
    char message[100];
    char response[200];
    mkfifo(SERVER_FIFO, 0666);
    mkfifo(CLIENT_FIFO, 0666);
    printf("Server started...\n");
    printf("Waiting for client messages...\n");

    while (1) {
        int server_fd = open(SERVER_FIFO, O_RDONLY);

        if (server_fd == -1) {
            perror("open");
            exit(1);
        }

        memset(message, 0, sizeof(message));

        read(server_fd, message, sizeof(message) - 1);
        close(server_fd);

        printf("Client Message: %s\n", message);

        if (strcmp(message, "exit") == 0) {
            printf("Server shutting down...\n");
            break;
        }

        snprintf(response, sizeof(response),
                 "Server processed: %s", message);

        int client_fd = open(CLIENT_FIFO, O_WRONLY);

        if (client_fd == -1) {
            perror("open");
            exit(1);
        }

        write(client_fd, response, strlen(response) + 1);
        close(client_fd);

        printf("Response sent to client.\n");
    }

    unlink(SERVER_FIFO);
    unlink(CLIENT_FIFO);
    return 0;
}
```

Client:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define SERVER_FIFO "server_fifo"
#define CLIENT_FIFO "client_fifo"

int main() {
    char message[100];
    char response[200];

    printf("Client started.\n");

    while (1) {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        int server_fd = open(SERVER_FIFO, O_WRONLY);

        if (server_fd == -1) {
            perror("open");
            exit(1);
        }

        write(server_fd, message, strlen(message) + 1);
        close(server_fd);

        if (strcmp(message, "exit") == 0)
            break;

        int client_fd = open(CLIENT_FIFO, O_RDONLY);

        if (client_fd == -1) {
            perror("open");
            exit(1);
        }

        memset(response, 0, sizeof(response));

        read(client_fd, response, sizeof(response) - 1);
        close(client_fd);

        printf("Server Response: %s\n", response);
    }

    return 0;
}
```

```
root@Rahul: ~/OSSP_FIFO_Sig x + v
root@Rahul:~# cd ~
mkdir OSSP_FIFO_Signal
cd OSSP_FIFO_Signal
root@Rahul:~/OSSP_FIFO_Signal# pwd
/root/OSSP_FIFO_Signal
root@Rahul:~/OSSP_FIFO_Signal# ls
root@Rahul:~/OSSP_FIFO_Signal# nano server1.c
root@Rahul:~/OSSP_FIFO_Signal# nano client1.c
root@Rahul:~/OSSP_FIFO_Signal# gcc server1.c -o server1
root@Rahul:~/OSSP_FIFO_Signal# gcc client1.c -o client1
root@Rahul:~/OSSP_FIFO_Signal# ls
client1 client1.c server1 server1.c
root@Rahul:~/OSSP_FIFO_Signal# |
```

```
root@Rahul: ~/OSSP_FIFO_Sig x + v
root@Rahul:~# cd ~
mkdir OSSP_FIFO_Signal
cd OSSP_FIFO_Signal
root@Rahul:~/OSSP_FIFO_Signal# pwd
/root/OSSP_FIFO_Signal
root@Rahul:~/OSSP_FIFO_Signal# ls
root@Rahul:~/OSSP_FIFO_Signal# nano server1.c
root@Rahul:~/OSSP_FIFO_Signal# nano client1.c
root@Rahul:~/OSSP_FIFO_Signal# gcc server1.c -o server1
root@Rahul:~/OSSP_FIFO_Signal# gcc client1.c -o client1
root@Rahul:~/OSSP_FIFO_Signal# ls
client1 client1.c server1 server1.c
root@Rahul:~/OSSP_FIFO_Signal# cd ~/OSSP_FIFO_Signal
root@Rahul:~/OSSP_FIFO_Signal# ./server
-bash: ./server: No such file or directory
root@Rahul:~/OSSP_FIFO_Signal# ./server1
Server started...
Waiting for client messages...
Client Message: Hello Server
Response sent to client.
Client Message: How are you?
Response sent to client.

root@Rahul:~# cd ~/OSSP_FIFO_Signal
./client1
Client started.
Enter message: Hello Server
Server Response: Server processed: Hello Server
Enter message: How are you?
Server Response: Server processed: How are you?
Enter message: |
```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal) {
    if (signal == SIGINT) {
        printf("\nSIGINT received. Interrupt signal handled.\n");
    }
    else if (signal == SIGTERM) {
        printf("\nSIGTERM received. Termination signal handled.\n");
    }
    else if (signal == SIGUSR1) {
        printf("\nSIGUSR1 received. User-defined signal handled.\n");
    }

    fflush(stdout);
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}

```

```

root@Rahul: ~
root@Rahul:~# nano signals.c
root@Rahul:~# gcc signals.c -o signals
root@Rahul:~# ./signals
Signal handling program started.
Process ID: 757
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

SIGUSR1 received. User-defined signal handled.
Program is running...

```

```

root@Rahul: ~
root@Rahul:~# kill -SIGUSR1 757
root@Rahul:~#

```